

Case Study — Gambit Hollow Cribbage

Exact scoring, complete matches and durable recovery in a two-seat club game

turing/gambit-hollow-cribbage | version 1.0.0 | 15 September 2026

The task

Build a full-stack cribbage club application whose board, server and SQLite records agree throughout a complete match. A correct 29-point hand is only the opening example: the app must also handle every pegging transition, score boundary, saved-state continuation and accepted retry without corrupting a game or the club ladder.

Area	Required behavior
Exact scoring	Forty supplied hand fixtures; overlapping fifteens, pairs and runs; flush/nobs rules; a live hand and pegging scoring bench with invalid-input rejection.
Complete cribbage match	Two seats, six-card deals, two discards each, hidden hands/crib, cut, legal pegging, go/31 resets, ordered show, alternating dealers and instant capped wins at 121.
Reproducible practice	Named deals and near-target starts exercise specific scoring boundaries, a seven-hand club match and continuation when one player has exhausted their cards.
Durable recovery	SQLite saved games and club ladder; revision checks, accepted-request identity binding, exact retry replay, lost-response UI recovery and two real process restarts.
Usable presentation	Inline SVG board/cards, real browser controls, keyboard focus, mobile reachability, pending-action feedback and six Visual assessment axes.

Evaluation design

Fifty-five criteria are reported across Render 2, Constraints 2, Functional 40, Polish 5 and Visual 6. Functional weights total 48. Judges use a shared real-browser playable-table prerequisite, then independent verdicts with deterministic practice journeys and requests discovered from the submitted app. Render/Constraints use all_pass gates; otherwise overall reward is 60% Functional, 20% Polish and 20% Visual.

Recorded outcomes

Run	Overall	Functional	Polish	Visual	Functional count
Oracle	0.9833	1.0000	1.0000	0.9167	40/40
GPT-5.4-mini (high)	0.6616	0.7083	0.6000	0.5833	29/40
Gemini 3.7 Flash (high)	0.8134	0.9167	0.4000	0.9167	37/40
Claude Haiku 4.5	0.0000	0.0000	0.0000	0.0000	0/40; interrupted
NOP	0.0000	0.0000	0.0000	0.0000	No-op control

Render and Constraints are 1.0 for Oracle, GPT and Gemini. Haiku has both gates zero. NOP is graded=0, no_op=1; its zeros are control outputs.

Oracle

Overall 0.9833, Functional 1.0000 with all 40 criteria passing, and Render/Constraints/Polish 1.0000. Visual 0.9167 reflects two raw 4/5 verdicts: spacing/layout and responsive consistency. Reasons cite blank spacing transitions, dense mobile controls and a shortened practice label. The required overall threshold passes, while the preferred Visual=1.0 authoring target is not reached.

GPT-5.4-mini

Overall 0.6616 is within the saved 0.1–0.7 range. Functional is 0.7083: 29/40 criteria passed, earning 34/48 weighted points. Eleven failures concern pegging/run/last-card calculations, a repeated discard accepted out of order, show totals, hand-state reset, match completion and lost-response retry after reload. Polish loses keyboard focus after a card selection and overflows at desktop/mobile widths. The basic working-table gate passed, making this more informative than the earlier two browser-blocked zero attempts.

Gemini 3.7 Flash

Overall 0.8134 is above 0.7. Functional is 0.9167, with 37/40 criteria passed and 44/48 weighted points. Failures concern invalid starting scores accepted/clamped, a duplicated physical card accepted by the pegging bench and loss of pending retry after reload. Polish loses mobile reachability, an explanatory illegal-move refusal and duplicate-action suppression. No separate Gemini acceptance band was found; this run would fail an all-model ceiling of 0.7.

Claude Haiku 4.5

The exported reward is 0.0000 with graded=1 and no_op=0, but the agent did not finish normally. UnknownApiError records command exit 143 immediately after `kill -f "node serve.js"`. The broad pattern likely also matched the agent command line, which contains the task instruction; self-termination is an inference, not a demonstrated process trace. The partial app returned "UI coming soon" from create/open controls. Every dimension assigned zero after the shared playable-table gate failed. Preserve this grade and rerun Haiku for clean completed-model evidence.

What the comparison supports

The Oracle demonstrates full recorded Functional coverage. GPT builds a playable, persistent application but loses correctness across linked scoring and match-state transitions. Gemini implements more of that behavior, while both model apps lose the pending retry after reload. This suggests that whole-match consistency and recovery expose gaps beyond individual scoring examples. Haiku cannot support the same comparison because its agent stopped early.

Interpretation limits

- This delivery packages existing supplied exports. It does not change task files, adjust recorded verdicts, rerun models or certify new platform QC.
- The saved GPT band applies to overall reward. Its Functional 0.7083 is slightly above 0.7, and the earlier aspirational overall target near 0.4 was not reached.
- The two Oracle Visual deductions remain in the report. Overall acceptance does not imply a perfect visual result.
- Haiku is interrupted evidence, not a normally completed attempt. Its 55 gate-zero verdicts are not 55 independently exercised implementation defects.
- Shared setup or gameplay failures can prevent later checkpoints. Exported failed criteria must not all be treated as independent root causes. This review did not replay every model browser action.
- Only one current attempt per model is included. These outcomes establish neither stable rankings nor repeatability across new runs.
- The supplied model set is GPT, Gemini and Haiku. No Sonnet export is available; Gemini is not relabeled as Sonnet. If the general checklist requiring Sonnet still applies, that evidence remains outstanding.
- No platform static/rubric QC report was supplied. Local archive/configuration checks are reported separately. The earlier fresh local environment build was blocked by the package proxy; no new local Docker build is claimed.

Delivery provenance

The task ZIP is byte-identical to the frozen browser-acceptance candidate. All 39 current task files match it; every golden solution file matches the Oracle app export. Every graded run contains the same 12 verifier hashes (five prompts, five judge files, runner and reward configuration). All five trials share the recorded task checksum and Harbor digest below.

Task ZIP SHA-256: 06a4f99349469c181d51b8da5520fa0c8bc02332c0433cbfc0d826b374e57f6a

Recorded task checksum: 770ea7287ee52452d6f44b7523b120e216b1a4d080969516efdeecdd8c70837c

Recorded Harbor task digest: sha256:ebe8a7bc5b75c6b6a8151c8cb985d85070042698e2888bc48ded8f6a7c230bb8

These three identifiers are different hash schemes and are not interchangeable. The platform checksums are reported from exports, not recomputed from the task ZIP. Original run exports are unchanged; archive copies preserve scores and verdicts and redact detected provider credentials if present. The Oracle archive includes its original NOP sibling.